

```
--- djangoapi\core\viewsKnoxLoginManagenent.py ---
```

```
# Create your views here.
```

```
#Django imports
```

```
from django.http import JsonResponse, HttpRequest
```

```
from django.contrib.auth.models import User
```

```
from django.contrib.auth.signals import user_logged_out
```

```
from django.contrib.sessions.models import Session
```

```
from django.contrib.auth.signals import user_logged_out
```

```
#rest framework imports
```

```
from rest_framework import permissions
```

```
from rest_framework.response import Response
```

```
from rest_framework.permissions import AllowAny, IsAuthenticated, IsAdminUser
```

```
from rest_framework import status
```

```
from rest_framework.views import APIView
```

```
from rest_framework.exceptions import AuthenticationFailed
```

```
from knox.views import LoginView as KnoxLoginView
```

```
from knox.models import AuthToken # Necesario para eliminar todos los tokens
```

```
from knox.auth import TokenAuthentication
```

```
#mis módulos
```

```
from core import serializers
```

```
from core.myLib import generalModule, drf, managePermissions
```

```
def notLoggedIn(request: HttpRequest):
```

```
    return JsonResponse({"ok":False,"message": "You are not logged in", "data":[]})
```

```
class IsValidToken(APIView):
```

```
    permission_classes=[permissions.IsAuthenticated]
```

```
    def post(self,request, format=None):
```

```
        user=request.user
```

```
        groups = managePermissions.getUserGroupsAsDict(user.username)
```

```
        os=AuthToken.objects.all().filter(user_id=user.id).count()
```

```
        return Response({
```

```
            'messages':{'exito':'Identificación realizada con éxito'},
```

```
            "politica_acceso":{"acceso":"Acceso permitido."},
```

```
            'data': [{"detail": "Token Válido.", "username": user.username, "user":  
user.id, "groups":groups, "opened_sessions":os}]})
```

```
class KnoxLogin(KnoxLoginView):
```

```
    permission_classes = (AllowAny, )
```

```
    serializer_class = serializers.LoginViewWithKnoxSerializer
```

```
    def post(self, request, format=None):
```

```
        """
```

```
        Operación login con usuario y contraseña.
```

```
        ---
```

```
        """
```

```

        serializer = self.serializer_class(data=request.data)
        valid= serializer.is_valid(raise_exception=False)#así, si no es válido, no lanza
excepción,

        #y se ejecuta el else:
        messages={}
        if not valid:
            mens=drf.manageSerializerErrors(serializer.errors)
            return Response({"messages": mens, "politica_acceso": {"acceso":"Acceso
abierto a la vista"},"data":None }, status=status.HTTP_401_UNAUTHORIZED)
        #print('Validated data')
        #print(serializer.validated_data)
        validated_data=serializer.validated_data #the attrs dictionary
        groups = managePermissions.getUserGroupsAsDict(request.data['username'])
        v={}
        v['groups'] = groups #la lista de grupos a los que pertenece el usuario
        v['username'] = request.data['username']
        os=AuthToken.objects.all().filter(user_id=validated_data['user'].id).count()
        v['opened_sessions']= os
        v["user"]= validated_data['user'].id
        v["token"]=validated_data['token']
        v["token_expiry"]=validated_data['token_expiry']
        messages['exito']='Success identification'
        messages['serializer_message']=validated_data['serializer_message']
        return Response({'messages':messages,"politica_acceso": {"acceso":"Acceso
abierto a la vista"},"data':[v]}, status=status.HTTP_200_OK)

class KnoxLogout(APIView):
    permission_classes = (IsAuthenticated, )

    def post(self,request, format=None):
        """
        Operación logout. Necesita el el campo Authorization en el header con el valor
Token XXXX....
        """
        user=request.user
        request.auth.delete()

        # Si existe una sesión de Django activa (gestionada por cookies/middleware)
        if hasattr(request, 'session') and request.session.session_key:
            request.session.flush() # Esto elimina la fila de public.django_session

        # 2. Envía la señal (usa 'user' y 'request' que ya tienes)
        user_logged_out.send(
            sender=user.__class__, # Usamos la clase del objeto user
            request=request,
            user=user # Usamos el objeto user autenticado
        )

        politica_msg = getattr(request, '_policy_log', {"politica_acceso": "Permiso
concedido"})
        return Response({
            "messages": {

```

```

        "exito": "Sesión cerrada."
    },
    "politica_acceso": politica_msg,
    'data': []
}, status=status.HTTP_200_OK)

```

```

class LogoutAllUserSessionsView(APIView):
    permission_classes = (IsAuthenticated, )
    serializer_class = serializers.LogoutAllUserSessionsSerializer
    def post(self, request, format=None):
        """
        Cierra todas las sesiones (tokens) de un usuario específico.
        Requiere el token actual en el header y el username en el cuerpo.
        - El token debe corresponder al usuario cuyo username se envía. En ese caso
        borra todas
        las sesiones, menos la actual.
        - Si lo anterior no pasa, si el usuario pertenece a un grupo con permisos
        especiales,
        ["admin"],
        se le permite cerrar las sesiones de otros usuarios.
        """

        # Obtener el username del cuerpo de la petición (POST data)
        posted_username = request.data.get('username')

        # Validar si falta el username
        if not posted_username:
            return Response({
                "messages": {"error_request": "The field 'username' is mandatory."},
                "access_policy": {"access": "Allowed."},
                "data": None
            }, status=status.HTTP_400_BAD_REQUEST)

        # Validar si el usuario solicitado existe
        l=list(User.objects.all().filter(username=posted_username))
        if len(l)==0:
            return Response(
                {
                    "messages": {"error_request": f"Wrong username: {posted_username}"},
                    "access_policy": {"access": "Allowed."},
                    "data": None
                },
                status=status.HTTP_404_NOT_FOUND
            )
        else:
            user_to_remove_sessions=User=l[0]

            user = request.user
            token_actual: AuthToken = request.auth

            os=AuthToken.objects.all().filter(user_id=user_to_remove_sessions.id).count()
            # Verificar que el usuario del token coincida con el usuario solicitado
            if user.username.lower() != posted_username.lower():

```

```

        men1="The posted username is not the authenticated user."
        if not user.groups.filter(name='admin').exists():
            men2="The user does not belong to a group to be able to delete other
user sessions."
            return Response({
                "messages": {"error_request": men1, "error_request_2": men2},
                "access_policy": {"access": "Access allowed to the view."},
                "data": None
            }, status=status.HTTP_403_FORBIDDEN) # 403 FORBIDDEN es más apropiado
aquí
        else:
            men2="The authenticated user belongs to a group to be able to delete
other user sessions."
            else:
                men1="The posted username is the authenticated user."
                if not user.groups.filter(name='admin').exists():
                    men2="The user does not belong to a group to be able to delete other
user sessions."
                else:
                    men2="The authenticated user belongs to a group to be able to delete
other user sessions."
            # -----
            # 3. Cierre de Sesiones (Lógica Principal)
            # -----

# 1. Eliminar TODOS los tokens de ese usuario
# Esto elimina todas las filas de KnoxAuthToken para el usuario autenticado

if men1 == "The posted username is the authenticated user.":
    if os==1:
        return Response({
            "messages": {
                "success": f"You only have one session opened. You can not close
it with this operation. Use logout instead.",
                "detail_1": men1,
                "detail_2": men2
            },
            "access_policy": {"access": "Access granted to the view."},
            "data": [{"username": posted_username, "closed_sessions": 0}]
        }, status=status.HTTP_200_OK)
    else:
        AuthToken.objects.filter(user=user).exclude(pk=token_actual.pk).delete()
        return Response({
            "messages": {
                "success": f"All your sessions, except the current one, have
been successfully closed. Username: {posted_username}. Closed sessions {os-1}",
                "detail_1": men1,
                "detail_2": men2
            },
            "access_policy": {"access": "Access granted to the view."},
            "data": [{"username": posted_username, "closed_sessions": os-1}]
        }, status=status.HTTP_200_OK)

```

```

else:
    AuthToken.objects.filter(user=user_to_remove_sessions).delete()

# 2. Eliminar TODAS las sesiones de Django para este usuario (¡CORRECCIÓN!)
# Esto busca todas las sesiones cuyo contenido serializado contiene el user_id
# Nota: SessionDataSerializer es una clase interna de Django para este propósito
all_sessions = Session.objects.all()

for session in all_sessions:
    try:
        # Obtener el objeto de sesión (que contiene 'user_id' si el usuario se
autenticó por sesión)
        session_data = session.get_decoded()

        # Comprobar si esta sesión pertenece al usuario actual
        if str(session_data.get('_auth_user_id')) ==
str(user_to_remove_sessions.pk):
            session.delete()
    except:
        # Ignorar sesiones mal formadas
        continue

# 3. Enviar la señal user_logged_out
user_logged_out.send(
    sender=user_to_remove_sessions.__class__,
    request=request,
    user=user_to_remove_sessions
)

# 4. Respuesta de éxito
return Response({
    "messages": {
        "success": f"All sessions ({os}) from the user {posted_username} have
been successfully closed.",
        "detail_1": men1,
        "detail_2": men2
    },
    "access_policy": {"access": "Access granted to the view."},
    "data": [{"username": posted_username, "closed_sessions": os}]
}, status=status.HTTP_200_OK)

class LogoutAllUsersSessionsView(APIView):
    permission_classes = (IsAdminUser, )
    serializer_class = serializers.EmptySerializer
    def post(self, request, format=None):
        """
        Cierra todas las sesiones (tokens) de todos los usuarios, excepto la sesión
actual.
        Requiere el token actual en el header y debe pertenecer a un grupo con permisos
especiales (admin),
        @param username: string en el body de la petición
        @return: Response
        """

```

```

user=request.user
token_actual = request.auth

# if not user.groups.filter(name="admin").exists():
#     men1="The user does not belong to the admin group."
#     return Response({
#         "messages": {"error_request": men1},
#         "access_policy": {"acceso": "Access granted to this view."},
#         "data": None
#     }, status=status.HTTP_403_FORBIDDEN) # 403 FORBIDDEN es más apropiado aquí

os=AuthToken.objects.all().count()

if os == 1:
    return Response({
        "messages": {
            "success": f"No other user sessions exist.",
            "detail_1": "The user belongs to the admin group."
        },
        "access_policy": {"access": "Access granted to this view."},
        "data": [{"username": user.username, "closed_sessions": 0}]
    }, status=status.HTTP_200_OK)

AuthToken.objects.all().exclude(pk=token_actual.pk).delete()

# 3. Enviar la señal user_logged_out
user_logged_out.send(
    sender=user.__class__,
    request=request,
    user=user
)

# 4. Respuesta de éxito
return Response({
    "messages": {
        "success": f"All sessions of all users, except the current one, have been closed: {os}.",
        "detail_1": "The user is superuser."
    },
    "access_policy": {"access": "Access granted to this view."},
    "data": [{"username": user.username, "closed_sessions": os-1}]
}, status=status.HTTP_200_OK)

```